

## Preparando los datos para el modelo

```
In [10]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
from sklearn.preprocessing import StandardScaler

plt.rcParams['figure.figsize'] = (12, 5)
sns.set_theme(style='whitegrid')

# Cargar y Limpiar – igual que Lección 2
df = pd.read_csv(
    '../datasets/household_power_consumption.txt',
    sep=';',
    na_values='?',
    low_memory=False
)

df['Datetime'] = pd.to_datetime(df['Date'] + ' ' + df['Time'],
                                format='%d/%m/%Y %H:%M:%S')
df.set_index('Datetime', inplace=True)
df.drop(columns=['Date', 'Time'], inplace=True)
df.dropna(inplace=True)

cols = ['Global_active_power', 'Global_reactive_power',
        'Voltage', 'Global_intensity',
        'Sub_metering_1', 'Sub_metering_2', 'Sub_metering_3']
df[cols] = df[cols].apply(pd.to_numeric, errors='coerce')
df.dropna(inplace=True)

print(df.shape)
```

(2049280, 7)

Ahora construyes las features. Aquí es donde el heatmap de la Lección 2 te sirve directamente:

```
In [11]: # Feature engineering – construir las variables de entrada
df['hora'] = df.index.hour
df['dia_semana'] = df.index.dayofweek
df['mes'] = df.index.month

# Features elegidas basadas en el heatmap de correlación
features = ['Global_intensity', 'Sub_metering_3',
            'Sub_metering_1', 'Sub_metering_2',
            'hora', 'dia_semana', 'mes']

X = df[features]
y = df['Global_active_power']

print(f"Features shape: {X.shape}")
print(f"Target shape: {y.shape}")
print(X.head())
```

Features shape: (2049280, 7)

Target shape: (2049280,)

| Datetime            | Global_intensity | Sub_metering_3 | Sub_metering_1 | \ |
|---------------------|------------------|----------------|----------------|---|
| 2006-12-16 17:24:00 | 18.4             | 17.0           | 0.0            |   |
| 2006-12-16 17:25:00 | 23.0             | 16.0           | 0.0            |   |
| 2006-12-16 17:26:00 | 23.0             | 17.0           | 0.0            |   |
| 2006-12-16 17:27:00 | 23.0             | 17.0           | 0.0            |   |
| 2006-12-16 17:28:00 | 15.8             | 17.0           | 0.0            |   |

| Datetime            | Sub_metering_2 | hora | dia_semana | mes |
|---------------------|----------------|------|------------|-----|
| 2006-12-16 17:24:00 | 1.0            | 17   | 5          | 12  |
| 2006-12-16 17:25:00 | 1.0            | 17   | 5          | 12  |
| 2006-12-16 17:26:00 | 2.0            | 17   | 5          | 12  |
| 2006-12-16 17:27:00 | 1.0            | 17   | 5          | 12  |
| 2006-12-16 17:28:00 | 1.0            | 17   | 5          | 12  |

### Train/Test split y escalado

```
In [12]: # Dividir en entrenamiento y prueba - 80/20
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print(f"Train: {X_train.shape[0]} filas")
print(f"Test: {X_test.shape[0]} filas")
```

Train: 1639424 filas

Test: 409856 filas

El random\_state=42 es una semilla que fija el resultado. Sin ella, cada vez que corras el código el split sería diferente y no podrías comparar resultados entre sesiones. Anótalo en tu cuaderno.

```
In [13]: # Escalado - normalizar las features al mismo rango
# La regresión lineal es sensible a las escalas
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Hay una regla crítica aquí que debes anotar en tu cuaderno: el scaler se entrena solo con X\_train, nunca con X\_test. Si escalas con todos los datos juntos, estás dejando que el modelo "vea" el futuro durante el entrenamiento. Eso se llama data leakage y es uno de los errores más graves en ML.

### Entrenar el modelo

```
In [14]: # Crear y entrenar el modelo
modelo = LinearRegression()
modelo.fit(X_train_scaled, y_train)

# Ver los coeficientes aprendidos
coeficientes = pd.DataFrame({
    'Feature': features,
    'Coeficiente': modelo.coef_
}).sort_values('Coeficiente', ascending=False)
```

```
print(coeficientes)
print(f"\nIntercepto: {modelo.intercept_:.4f}")
```

|   | Feature          | Coeficiente |
|---|------------------|-------------|
| 0 | Global_intensity | 1.042781    |
| 1 | Sub_metering_3   | 0.021391    |
| 4 | hora             | 0.001292    |
| 2 | Sub_metering_1   | -0.000694   |
| 5 | dia_semana       | -0.000745   |
| 6 | mes              | -0.001112   |
| 3 | Sub_metering_2   | -0.001824   |

Intercepto: 1.0917

Los coeficientes son lo que el modelo aprendió. Un coeficiente alto positivo significa que cuando esa variable sube, el consumo predicho sube. Un coeficiente negativo significa lo contrario. Compara los coeficientes con lo que el heatmap te mostró: ¿coinciden con las correlaciones que viste?

### Evaluar el modelo

```
In [15]: # Predecir sobre datos que el modelo nunca vio
y_pred = modelo.predict(X_test_scaled)

# Métricas de evaluación
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
mae = mean_absolute_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print(f"RMSE: {rmse:.4f} kW")
print(f"MAE: {mae:.4f} kW")
print(f"R²: {r2:.4f}")
```

```
RMSE: 0.0464 kW
MAE: 0.0312 kW
R²: 0.9981
```

Anota en tu cuaderno qué mide cada métrica:

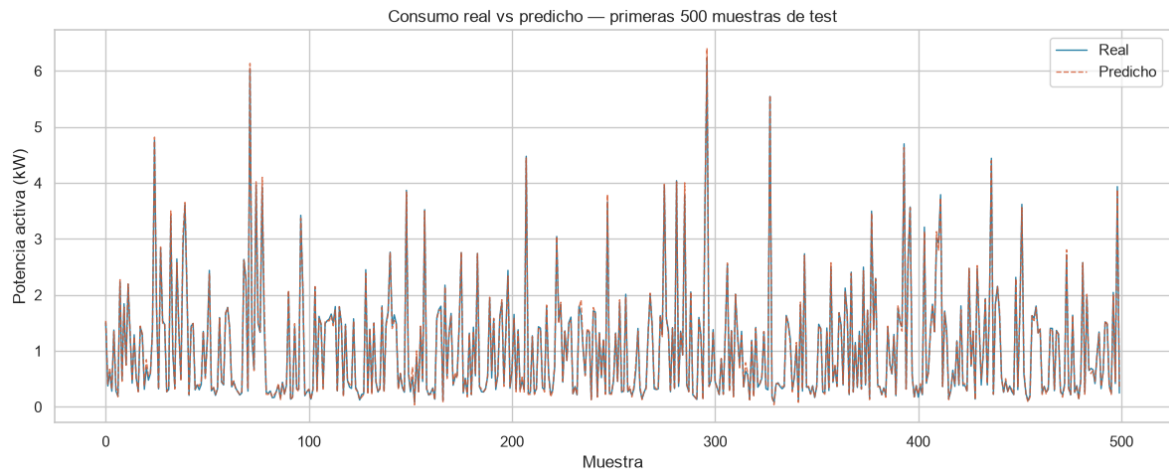
MAE (Mean Absolute Error): el error promedio de tus predicciones en las mismas unidades del target. Si el MAE es 0.3, el modelo se equivoca en promedio 0.3 kW.

RMSE (Root Mean Squared Error): similar al MAE pero penaliza más los errores grandes. Siempre es mayor o igual al MAE.

R<sup>2</sup> (R cuadrado): qué porcentaje de la variación del target explica tu modelo. Un R<sup>2</sup> de 0.85 significa que el modelo explica el 85% de la variación del consumo. Más cercano a 1 es mejor.

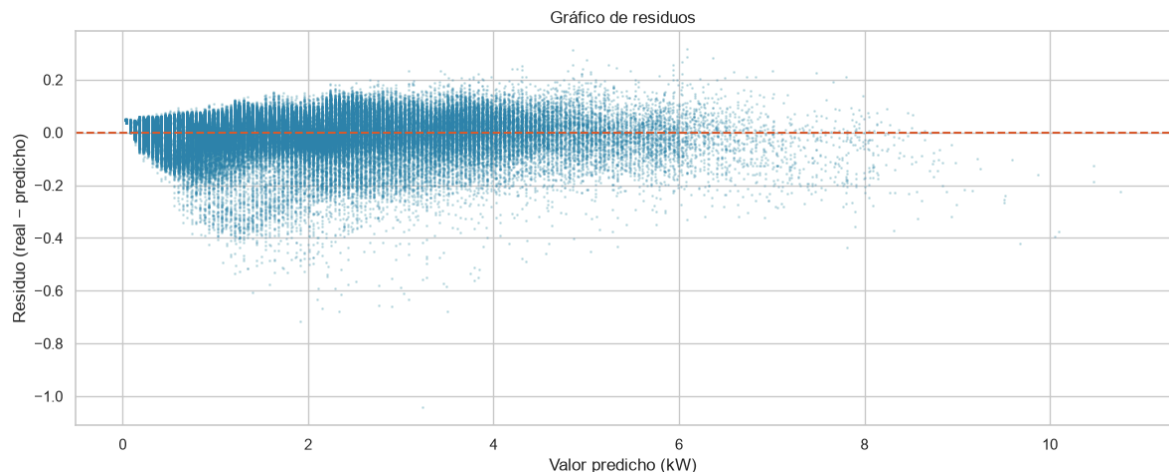
```
In [16]: # Visualizar predicciones vs valores reales
muestra = 500
fig, ax = plt.subplots()
ax.plot(y_test.values[:muestra], label='Real', color='#2E86AB', linewidth=1)
ax.plot(y_pred[:muestra], label='Predicho', color='#D85A30',
        linewidth=1, alpha=0.8, linestyle='--')
```

```
ax.set_title('Consumo real vs predicho – primeras 500 muestras de test')
ax.set_xlabel('Muestra')
ax.set_ylabel('Potencia activa (kW)')
ax.legend()
plt.tight_layout()
plt.show()
```



```
In [17]: # Gráfico de residuos – diferencia entre real y predicho
residuos = y_test.values - y_pred

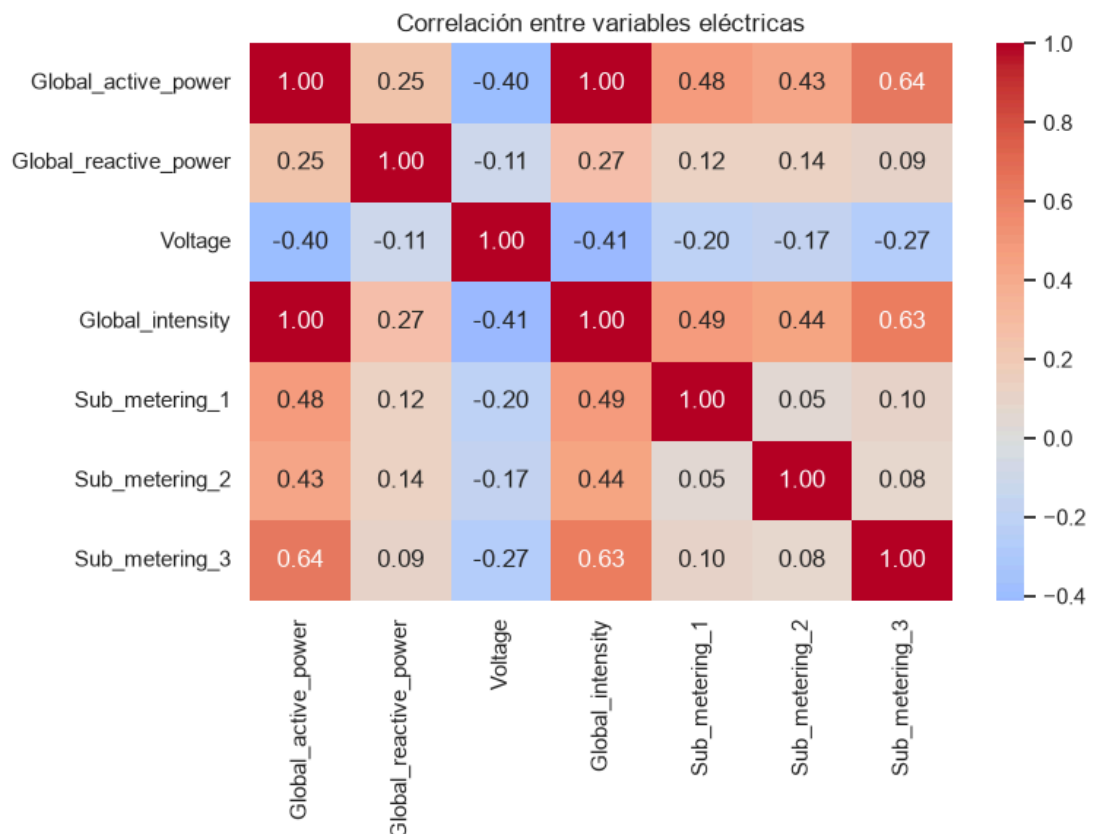
fig, ax = plt.subplots()
ax.scatter(y_pred, residuos, alpha=0.2, color='#2E86AB', s=1)
ax.axhline(y=0, color='#D85A30', linewidth=1.5, linestyle='--')
ax.set_title('Gráfico de residuos')
ax.set_xlabel('Valor predicho (kW)')
ax.set_ylabel('Residuo (real - predicho)')
plt.tight_layout()
plt.show()
```



El gráfico de residuos es clave: si los puntos están distribuidos aleatoriamente alrededor de la línea cero, el modelo es bueno. Si ves un patrón (una curva, un embudo), el modelo tiene un problema estructural y necesitas uno más complejo.

Se puede observar que el gráfico de residuo tiene una forma no aleatoria, por el contrario, tiene una forma pegada al cero, lo que da un indicio de un problema estructural. Asimismo, los valores de  $R^2$  tiene un valor muy alto, lo que tiende a pensar que hay un problema a nivel de features.

Luego de ver el siguiente diagrama:



Se puede ver que Global intensidad tiene correlación perfecta de 1 con la variable de salida, lo que ocasionó que el modelo encontrara la correlación lineal entre ambos parámetros y tomara el camino corto prediciendo el valor de salida obtenido de su variable de entrada. A continuación, se hará un entrenamiento de un modelo pero eliminando la variable linealmente dependiente.

### eliminamos el feature que tenía correlación lineal perfecta

```
In [18]: # Feature engineering – construir las variables de entrada
df['hora'] = df.index.hour
df['dia_semana'] = df.index.dayofweek
df['mes'] = df.index.month

# Features elegidas basadas en el heatmap de correlación
features = ['Sub_metering_3',
            'Sub_metering_1', 'Sub_metering_2',
            'hora', 'dia_semana', 'mes']

X = df[features]
y = df['Global_active_power']

print(f"Features shape: {X.shape}")
print(f"Target shape: {y.shape}")
print(X.head())
```

Features shape: (2049280, 6)

Target shape: (2049280,)

|                     | Sub_metering_3 | Sub_metering_1 | Sub_metering_2 | hora | \ |
|---------------------|----------------|----------------|----------------|------|---|
| Datetime            |                |                |                |      |   |
| 2006-12-16 17:24:00 | 17.0           | 0.0            | 1.0            | 17   |   |
| 2006-12-16 17:25:00 | 16.0           | 0.0            | 1.0            | 17   |   |
| 2006-12-16 17:26:00 | 17.0           | 0.0            | 2.0            | 17   |   |
| 2006-12-16 17:27:00 | 17.0           | 0.0            | 1.0            | 17   |   |
| 2006-12-16 17:28:00 | 17.0           | 0.0            | 1.0            | 17   |   |

|                     | dia_semana | mes |
|---------------------|------------|-----|
| Datetime            |            |     |
| 2006-12-16 17:24:00 | 5          | 12  |
| 2006-12-16 17:25:00 | 5          | 12  |
| 2006-12-16 17:26:00 | 5          | 12  |
| 2006-12-16 17:27:00 | 5          | 12  |
| 2006-12-16 17:28:00 | 5          | 12  |

```
In [19]: # Dividir en entrenamiento y prueba - 80/20
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print(f"Train: {X_train.shape[0]} filas")
print(f"Test: {X_test.shape[0]} filas")
```

Train: 1639424 filas

Test: 409856 filas

```
In [20]: # Escalado - normalizar las features al mismo rango
# La regresión lineal es sensible a las escalas
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

### Entrenamos el modelo

```
In [21]: # Crear y entrenar el modelo
modelo = LinearRegression()
modelo.fit(X_train_scaled, y_train)

# Ver los coeficientes aprendidos
coeficientes = pd.DataFrame({
    'Feature': features,
    'Coeficiente': modelo.coef_
}).sort_values('Coeficiente', ascending=False)

print(coeficientes)
print(f"\nIntercepto: {modelo.intercept_:.4f}")
```

|   | Feature        | Coeficiente |
|---|----------------|-------------|
| 0 | Sub_metering_3 | 0.584192    |
| 1 | Sub_metering_1 | 0.413208    |
| 2 | Sub_metering_2 | 0.374067    |
| 3 | hora           | 0.153268    |
| 4 | dia_semana     | 0.028179    |
| 5 | mes            | -0.009715   |

Intercepto: 1.0917

## Evaluamos nuevamente el modelo

```
In [22]: # Predecir sobre datos que el modelo nunca vio
y_pred = modelo.predict(X_test_scaled)

# Métricas de evaluación
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
mae = mean_absolute_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print(f"RMSE: {rmse:.4f} kW")
print(f"MAE: {mae:.4f} kW")
print(f"R²: {r2:.4f}")
```

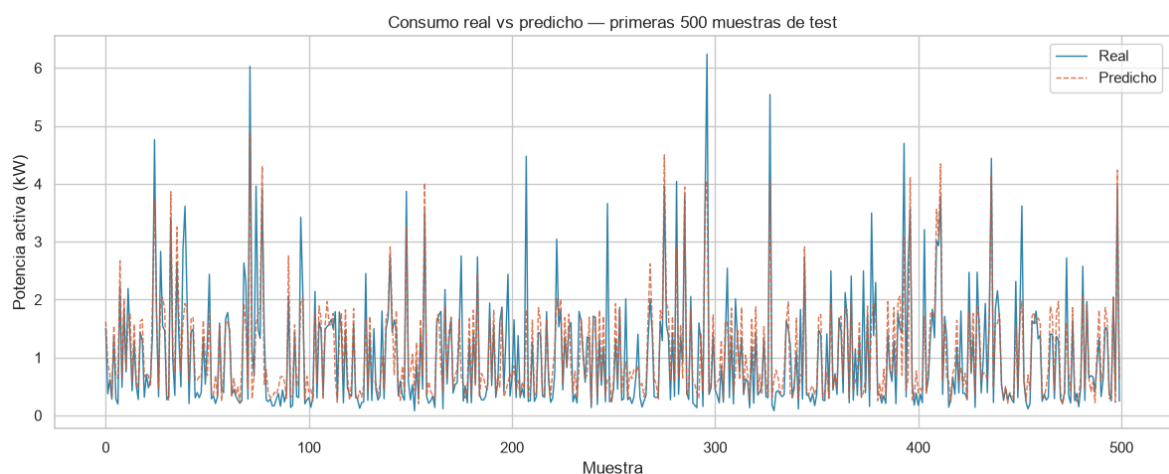
RMSE: 0.5379 kW

MAE: 0.3557 kW

R²: 0.7426

## Gráfico de predicción - residuo

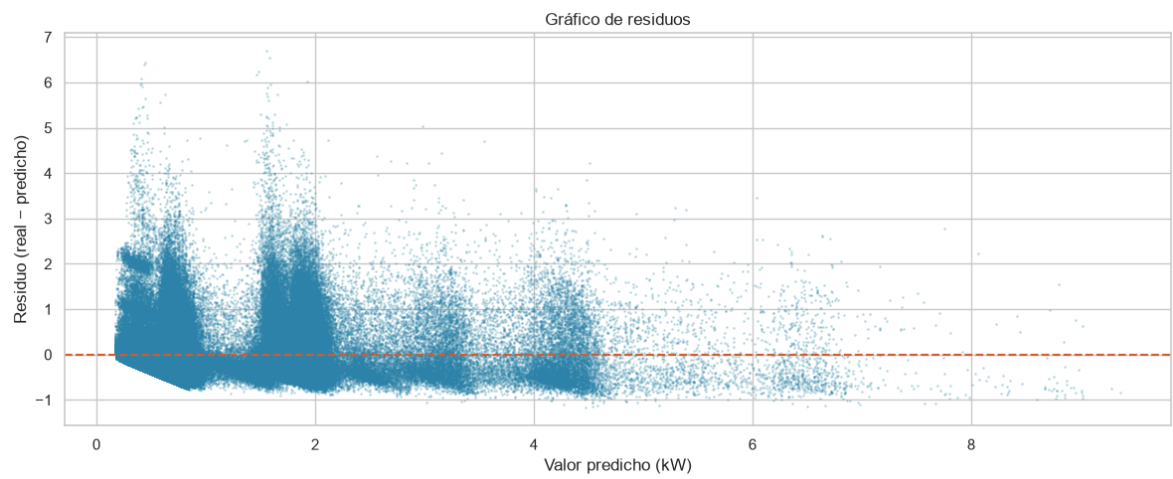
```
In [23]: # Visualizar predicciones vs valores reales
muestra = 500
fig, ax = plt.subplots()
ax.plot(y_test.values[:muestra], label='Real', color='#2E86AB', linewidth=1)
ax.plot(y_pred[:muestra], label='Predicho', color='#D85A30',
        linewidth=1, alpha=0.8, linestyle='--')
ax.set_title('Consumo real vs predicho – primeras 500 muestras de test')
ax.set_xlabel('Muestra')
ax.set_ylabel('Potencia activa (kW)')
ax.legend()
plt.tight_layout()
plt.show()
```



```
In [24]: # Gráfico de residuos – diferencia entre real y predicho
residuos = y_test.values - y_pred

fig, ax = plt.subplots()
ax.scatter(y_pred, residuos, alpha=0.2, color='#2E86AB', s=1)
ax.axhline(y=0, color='#D85A30', linewidth=1.5, linestyle='--')
ax.set_title('Gráfico de residuos')
ax.set_xlabel('Valor predicho (kW)')
ax.set_ylabel('Residuo (real - predicho)')
```

```
plt.tight_layout()  
plt.show()
```



In [ ]: